Software Engineering II

Workshop 2



CocoaTeaDevs:

Julian Ramirez Diaz

27/10/2026

SpillTheStats: Design Workshop

SpillTheStats is a web platform that centralizes League of Legends statistics for LAN players, offering organized builds and personalized recommendations based on each user's game history.

This document contains the four deliverables of the workshop: the system domain, the CRC cards, the UML class diagram and the relational database model. A short architecture overview is added because the system is built as a serverless JAMstack application.

Item	Decision
Scope (MVP)	Ranked Solo/Duo, LAN server, last 20 games. Champion recommendations are the minimum (US-03); items and runes are part of the base experience.
Target players	Players between Bronze and Emerald with 1 to 4 years of experience who want to climb.
Core idea	The player's games are turned into playstyle **tags** (e.g., Skirmisher, Duelist). A mapping table connects tags to champions, items and runes, and that match produces the recommendation.
Plans	Free: champions, items and runes. Premium: same, plus a deeper analysis of the matches (not yet defined; used as a base assumption).
Technology	React + Vite + Bootstrap 5 (frontend), Supabase (PostgreSQL, Auth, Storage), Vercel (hosting and serverless functions), Riot Games API and Data Dragon.

Contents

- 1. System domain: context, actors, business rules
- 2. CRC cards
- 3. Class diagram
- 4. Relational database model (ER diagram and table definitions)
- 5. Architecture overview
- 6. Checklist and references

1. System Domain

1.1 System context

SpillTheStats reads a player's last ~20 Ranked Solo/Duo games through the Riot API, derives the playstyle as scored tags, and matches those tags against a mapping table of champions, items and rune paths for the current patch. Users need an account because the platform has a Free and a Premium plan.

Inside the system boundary	Outside the system boundary (external)
Web frontend (React + Vite + Bootstrap 5) Serverless functions on Vercel (call Riot, run the algorithm) Supabase: PostgreSQL, Auth, Storage	**Riot Games API**: ACCOUNT-V1, MATCH-V5 (LAN uses platform la1, region americas) **Data Dragon**: champions, items, runes, images and the patch version

Main flow: log in, link Riot ID, request analysis, the server fetches matches, metrics become tags, tags are matched to the mapping table, recommendations are shown.

1.2 Actor characterization

Actor	Type	Goal	Main interactions
Free Player	Human	Get recommendations to improve and climb	Register, log in, link Riot ID, request analysis, view champions, items and runes, see inconclusive warnings
Premium Player	Human (specialization of Player)	Get a deeper analysis of own matches	Everything a Free Player does, plus the deeper match analysis (more metrics per game, more detailed playstyle breakdown)
Administrator	Human	Keep game data and tag rules accurate	Trigger the patch / static-data update, edit tag mapping tables, manage users and plans. No access to passwords
Riot Games API	External system	Source of account and match data	Answers account and match queries. Rate-limited and key-protected
Data Dragon	External system	Source of static data and images	Provides the current patch version and static JSON / images. Public, read-only

Player is a generalization: Free Player and Premium Player share the same base behavior.

1.3 Business rules

ID	Rule
Accounts and access	
BR-01	An account requires a valid email and a password of at least 8 characters (US-01).
BR-02	Only authenticated users can request analyses or view recommendations.
BR-03	A user links one Riot ID (Name#TAG) in the LAN region. It is saved only after the Riot API confirms it exists (US-02).
BR-04	Each Player sees only their own data. Only Administrators modify static data and tag mappings.
BR-05	Free and Premium both get champions, items and runes. Premium unlocks a deeper analysis of the matches (more metrics per game, e.g., early-game data, and a more detailed playstyle breakdown).
Data collection and analysis	
BR-06	Only Ranked Solo/Duo games are analyzed. Remakes and games shorter than ~5 minutes are excluded.
BR-07	Up to the last 20 eligible games are analyzed. There is no minimum number of games.
BR-08	If there are few games or the results are too mixed, results are still shown with a warning that they may not be conclusive (e.g., fewer than 10 games, or no clearly dominant tag).

ID	Rule
BR-09	Playstyle is a set of tags, each with a score from 0 to 1, derived from the match metrics.
BR-10	Only metrics necessary for tagging are used: KDA, CS/min, damage share, kill participation, vision score, objectives.
Recommendation	
BR-11	Recommendations come from the tag mapping table: every champion, item and rune carries tags with a weight; the score is how well they match the player's tags.
BR-12	Each result has at least 3 champions (US-03), an ordered item build with situational items, and one rune path.
BR-13	Recommendations are recalculated every time the Player requests an update.
BR-14	Only data from the current patch is used. Older patches are not available.
Riot API compliance	
BR-15	The Riot API key lives only on the server (Vercel environment variable), never in the browser.
BR-16	Riot rate limits are respected. At ~80% of the limit the optimization protocol activates (caching, queueing or throttling, with a clear message to the user).
BR-17	The site shows Riot's required disclaimer (not endorsed by Riot Games) and follows Riot's data policies.
Static data and privacy	
BR-18	Static data (champions, items, runes, images) is stored in the database. An Administrator-triggered update replaces it when Data Dragon reports a new patch. Only one patch is kept.
BR-19	Beyond login credentials only what is strictly necessary is stored: Riot ID, PUUID, region, plan, and the latest computed profile and recommendation. Raw match data is never stored.
BR-20	A Player can delete their account and all associated data.

2. CRC Cards

The model has **15 classes**: 10 domain classes and 5 technical classes. The technical classes are the minimum a serverless system needs (Riot access, rate limiting, analysis, recommendation, data sync). **User** covers both Player and Administrator through a *role* and a *plan*, so no subclasses are needed. **Tag** and **TagMapping** are the heart of the algorithm.

Domain classes

User domain	
Account holder. A Player (Free or Premium) or an Administrator.	
Responsibilities	**Collaborators**
• Hold identity data (email, role: Player/Administrator, plan: Free/Premium) • Register and authenticate (delegated to Supabase Auth) • Link and unlink one Riot account • Request an analysis and view own recommendations • Decide feature access by plan (e.g., deep analysis) • Administrator role: trigger static-data update, edit tag mappings • Delete own account and data	• RiotProfile • Recommendation • PlaystyleAnalyzer • StaticDataSyncService • TagMapping

RiotProfile domain

The Riot account linked to a user.

Responsibilities	Collaborators
• Store Riot ID (Name#TAG), PUUID and region (LAN) • Store verification status and last analysis date • Provide the PUUID needed to fetch matches	• User • RiotApiClient • PlaystyleProfile

MatchStats domain, transient

Metrics of one ranked game. Exists only during an analysis.

Responsibilities	Collaborators
• Represent one game's metrics (KDA, CS/min, damage share, kill participation, vision, objectives, result, champion) • Flag whether the game is eligible (Ranked Solo/Duo, not a remake, longer than 5 min) • Never persisted	• RiotApiClient • PlaystyleAnalyzer

PlaystyleProfile domain

Result of the analysis: the player's playstyle as scored tags.

Responsibilities	Collaborators
• Hold the tags with a score each (e.g., Skirmisher 0.7) • Record how many games were analyzed • Say whether the result is conclusive or needs a warning (BR-08) • Provide the dominant tags to the engine	• PlaystyleAnalyzer • Tag • RiotProfile • RecommendationEngine

Tag domain

A playstyle label (Skirmisher, Duelist, ...). The shared vocabulary of the system.

Responsibilities	Collaborators
• Define a label with name and description • Connect playstyle and game entities through one vocabulary • Be editable by an Administrator	• TagMapping • PlaystyleProfile

TagMapping domain

Knowledge base of the algorithm: how well a tag fits a champion, item or rune path.

Responsibilities	Collaborators
• Link a Tag to a Champion, Item or RunePath with a weight • Be maintained by an Administrator and reviewed on each patch	• Tag • Champion • Item • RunePath • RecommendationEngine

Champion domain

A champion of the current patch.

Responsibilities	Collaborators
• Hold static data (name, roles, base stats, image) • Be imported from Data Dragon • Be a candidate for champion recommendations	• TagMapping • StaticDataSyncService • Recommendation

Item domain	
An item of the current patch.	
Responsibilities	**Collaborators**
• Hold static data (name, cost, stats, image) • Be classified as core or situational in a build • Be a candidate for item recommendations	• TagMapping • StaticDataSyncService • Recommendation

RunePath domain	
A rune setup (tree, keystone, runes) of the current patch.	
Responsibilities	**Collaborators**
• Hold the rune trees, keystone and selected runes • Be imported from Data Dragon • Be a candidate for the recommended rune path	• TagMapping • StaticDataSyncService • Recommendation

Recommendation domain	
The result of one request. Replaced on every update.	
Responsibilities	**Collaborators**
• Hold at least 3 champions, an ordered item list with situational items, and one rune path • Store the warning, the patch version and the generation date • Be regenerated on every update request (BR-13)	• RecommendationEngine • User • Champion • Item • RunePath • PlaystyleProfile

Technical classes

RiotApiClient technical (Adapter)	
Single door to the Riot API.	
Responsibilities	**Collaborators**
• Resolve Riot ID to PUUID (ACCOUNT-V1) • Fetch match list and details (MATCH-V5) • Keep the API key server-side only (BR-15) • Translate API errors (not found, 429, timeouts) into clear messages	• RiotProfile • MatchStats • RateLimiter

RateLimiter technical	
Protects the Riot rate limits.	
Responsibilities	**Collaborators**
• Count requests against Riot's limits • Activate the optimization protocol at ~80% (queue, delay or throttle) (BR-16) • Give a "try again in X seconds" signal to the user	• RiotApiClient

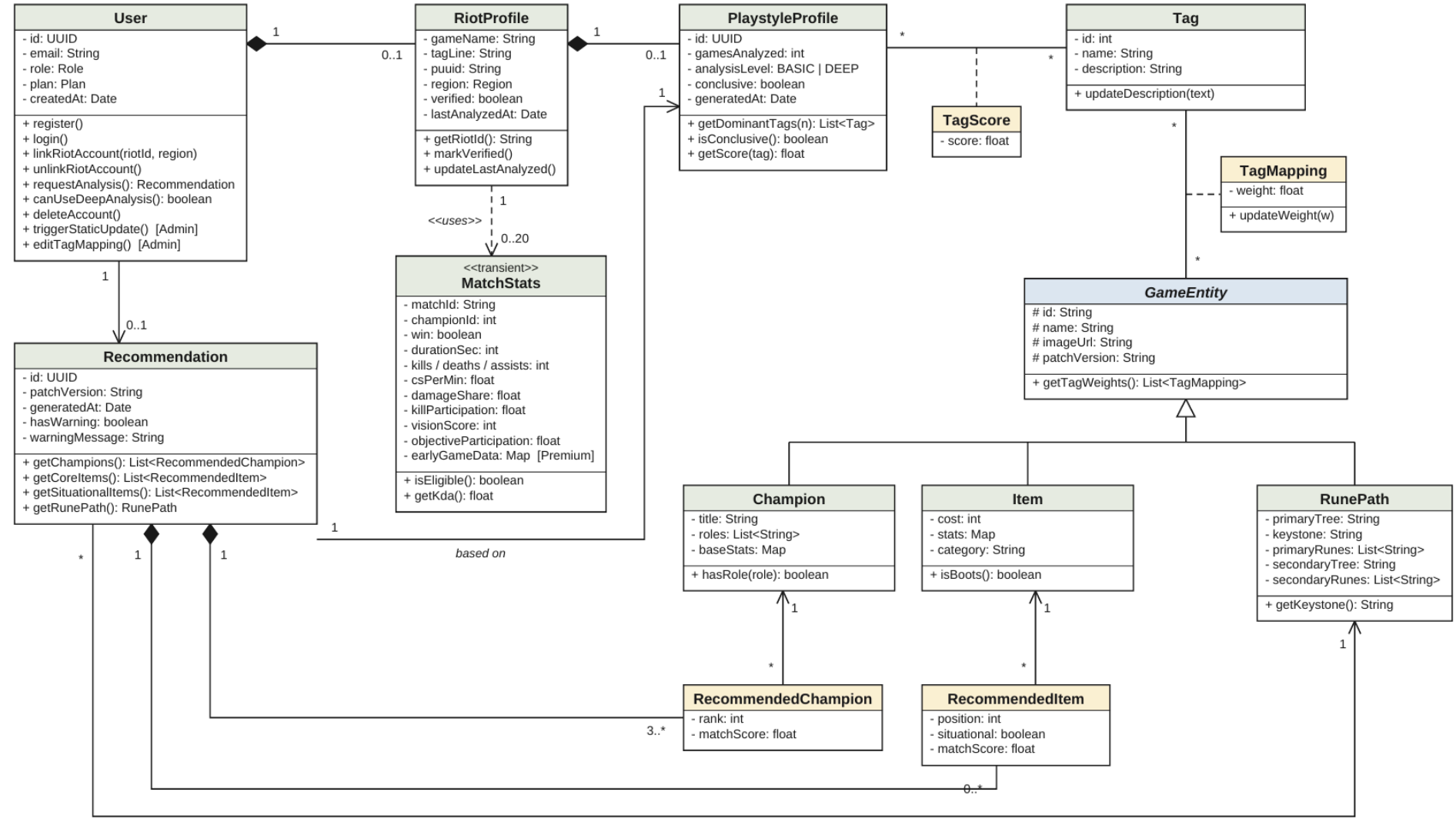
PlaystyleAnalyzer technical	
Turns match metrics into tag scores.	
Responsibilities	**Collaborators**
• Convert the list of MatchStats into tag scores • Apply the few-data / mixed-result check (BR-08) • Run the deeper analysis only for Premium users	• MatchStats • PlaystyleProfile • Tag • User

RecommendationEngine technical	
Matches the player's tags with the knowledge base.	
Responsibilities	**Collaborators**
• Match the player's tags against TagMapping weights • Rank champions, items (core and situational) and rune paths • Build the final Recommendation using current-patch data only	• PlaystyleProfile • TagMapping • Recommendation • Champion, Item, RunePath

StaticDataSyncService technical	
Keeps the static game data on the current patch.	
Responsibilities	**Collaborators**
• Check the current patch version on Data Dragon • Import champions, items, runes and images, replacing the previous patch (BR-18) • Run only when an Administrator triggers it • Report the result (success/failure, version)	• Data Dragon (external) • Champion • Item • RunePath • User (Administrator)

Main collaboration: **User** requests an analysis → **RiotApiClient** (through **RateLimiter**) fetches **MatchStats** → **PlaystyleAnalyzer** produces a **PlaystyleProfile** → **RecommendationEngine** uses **TagMapping** to build a **Recommendation** → shown to the **User**.

3. Class Diagram (domain model)



Visibility: - private, + public, # protected. Yellow = association classes / result items; blue = abstract class.

Class Diagram: how to read it

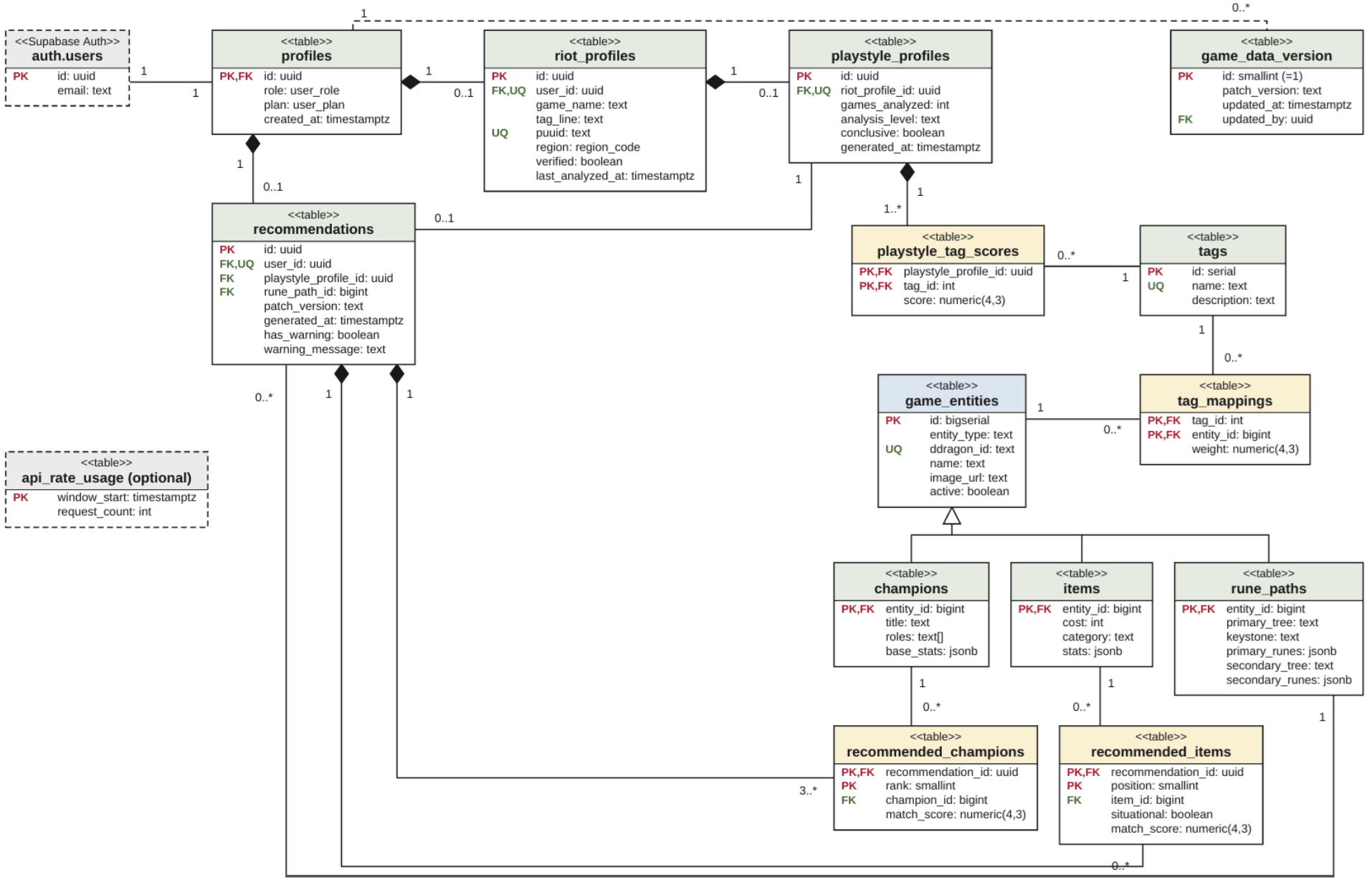
The diagram shows the domain model only. The technical classes of the CRC section (RiotApiClient, RateLimiter, PlaystyleAnalyzer, RecommendationEngine, StaticDataSyncService) belong to the architecture. Three small classes were added to make the UML correct: **GameEntity** (abstract parent of Champion, Item and RunePath), **TagScore** and **RecommendedChampion / RecommendedItem**.

From	To	Relationship	Multiplicity	Meaning
User	RiotProfile	Composition	1 to 0..1	Owns at most one linked Riot profile
User	Recommendation	Association	1 to 0..1	Latest recommendation, overwritten on update (BR-13)
RiotProfile	PlaystyleProfile	Composition	1 to 0..1	Latest computed profile
RiotProfile	MatchStats	Dependency	1 to 0..20	Matches are used temporarily, never stored
PlaystyleProfile	Tag	Association class: TagScore	* to *	Score of each tag in a profile
Tag	GameEntity	Association class: TagMapping	* to *	Knowledge base of the algorithm
Champion, Item, RunePath	GameEntity	Generalization	-	The three are subtypes
Recommendation	PlaystyleProfile	Association	1 to 1	Profile the result is based on
Recommendation	RecommendedChampion	Composition	1 to 3..*	At least 3 champions (US-03)
Recommendation	RecommendedItem	Composition	1 to 0..*	Ordered list, core and situational
Recommendation	RunePath	Association	* to 1	One recommended rune path
RecommendedChampion / RecommendedItem	Champion / Item	Association	* to 1	Which entity was recommended

Design decisions

- **MatchStats is transient**: it is used during the analysis and never stored, so it appears as a dependency (BR-19).
- **Role and plan are attributes of User** instead of subclasses, to keep the model simple. Splitting into Player and Administrator is a possible later refactor.
- **GameEntity** lets one TagMapping structure serve champions, items and runes.
- **Premium deep analysis** appears only as *analysisLevel* and the optional *earlyGameData*, because the team has not defined it yet.
- Business logic (tagging, scoring, rate limiting) lives in the technical services, so the domain methods are intentionally light.

4. Relational Database Model (ER Diagram, UML style)



Legend PK primary key FK foreign key UQ unique

Owner -> child (ON DELETE CASCADE) Supertype / subtype (shared PK) Foreign-key relationship Optional FK (updated_by)

Yellow = association tables; blue = supertype. MatchStats has no table (transient). One patch only: static data is upserted by Data Dragon ID. RLS is enabled on every table.

Relational Database Model: table definitions

The schema is for **PostgreSQL on Supabase**, in **third normal form**, and stores only the current patch.

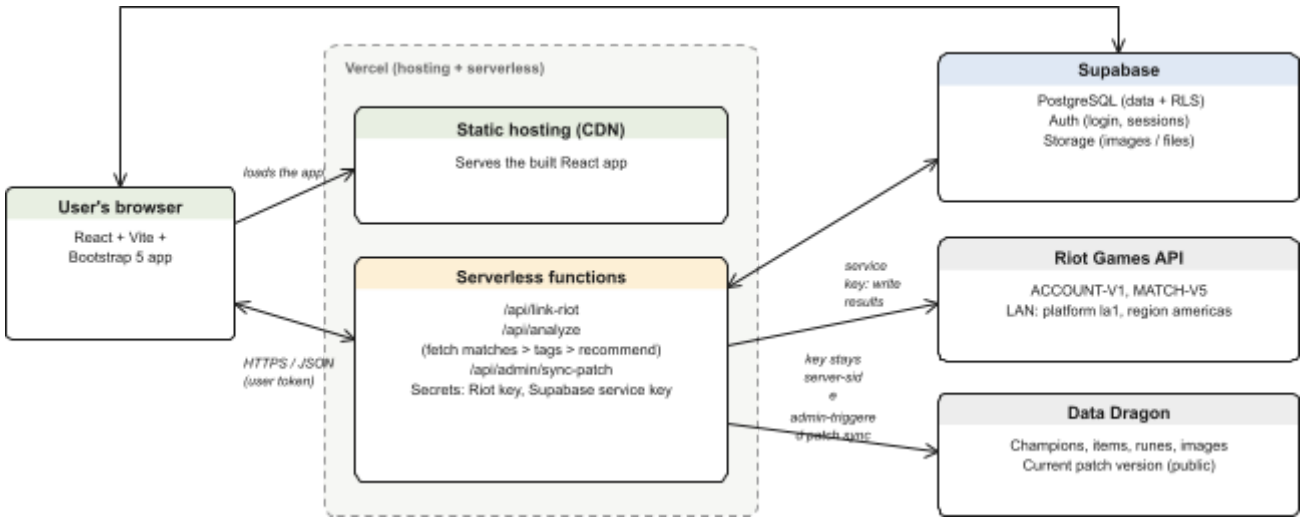
- **Login data is not duplicated**: email and password stay in Supabase Auth (*auth.users*); *profiles* only adds role and plan with the same UUID.
- **MatchStats has no table**: raw match data is never persisted. Only the computed profile and recommendation are kept.
- **GameEntity becomes a supertype table** with three subtype tables sharing its primary key, so one *tag_mappings* table can point to any of them with a real foreign key.
- **Easy patch update**: the update job upserts by Data Dragon ID. Tag mappings reference our own *entity_id*, so the administrator's work survives every patch. Removed entities are marked inactive; new ones appear without tags (a short "to review" list).
- **Latest result only**: *playstyle_profiles* and *recommendations* are unique per owner, so each update overwrites the previous result (BR-13).
- **Security**: Row Level Security on every table. Players read and write only their own rows; static tables are writable only by administrators. Writes from serverless functions use the Supabase service key, kept server-side.
- Columns typed *jsonb* or *text[]* hold read-only Data Dragon payloads that are never queried field by field, so they are not split into more tables.

Table	Columns	Notes
profiles	**id** uuid PK, FK → auth.users role user_role (PLAYER, ADMINISTRATOR) plan user_plan (FREE, PREMIUM) created_at timestamptz	Extends Supabase Auth; no password stored here
riot_profiles	**id** uuid PK user_id uuid FK → profiles, UQ game_name, tag_line text puuid text UQ region region_code (LAN) verified boolean last_analyzed_at timestamptz	One linked account per user
playstyle_profiles	**id** uuid PK riot_profile_id uuid FK → riot_profiles, UQ games_analyzed int (0 to 20) analysis_level text (BASIC, DEEP) conclusive boolean generated_at timestamptz	Latest profile only
playstyle_tag_scores	**playstyle_profile_id** uuid PK, FK **tag_id** int PK, FK → tags score numeric(4,3) (0 to 1)	Association class TagScore
tags	**id** serial PK name text UQ description text	Playstyle vocabulary
game_entities	**id** bigserial PK entity_type text (CHAMPION, ITEM, RUNE_PATH) ddragon_id text, UQ with entity_type name, image_url text active boolean	Supertype. Upsert by Data Dragon ID; removed entities are set inactive
champions	**entity_id** bigint PK, FK → game_entities title text, roles text[], base_stats jsonb	Subtype
items	**entity_id** bigint PK, FK → game_entities cost int, category text, stats jsonb	Subtype
rune_paths	**entity_id** bigint PK, FK → game_entities primary_tree, keystone text primary_runes jsonb secondary_tree text, secondary_runes jsonb	Subtype

Table	Columns	Notes
tag_mappings	**tag_id** int PK, FK → tags **entity_id** bigint PK, FK → game_entities weight numeric(4,3) (0 to 1)	Association class TagMapping; survives patch updates
game_data_version	**id** smallint PK (CHECK id = 1) patch_version text updated_at timestamptz updated_by uuid FK → profiles	Single row: the patch stored
recommendations	**id** uuid PK user_id uuid FK → profiles, UQ playstyle_profile_id uuid FK rune_path_id bigint FK → rune_paths patch_version text, generated_at timestamptz has_warning boolean, warning_message text	Latest result per user
recommended_champions	**recommendation_id** uuid PK, FK **rank** smallint PK champion_id bigint FK → champions match_score numeric(4,3)	At least 3 rows (enforced by the app)
recommended_items	**recommendation_id** uuid PK, FK **position** smallint PK (build order) item_id bigint FK → items situational boolean match_score numeric(4,3)	Core and situational items
api_rate_usage (optional)	**window_start** timestamptz PK request_count int	Shared state for the 80% protocol (stateless functions)

PK primary key, FK foreign key, UQ unique. Deleting a row in profiles cascades through all the user's data (BR-20)

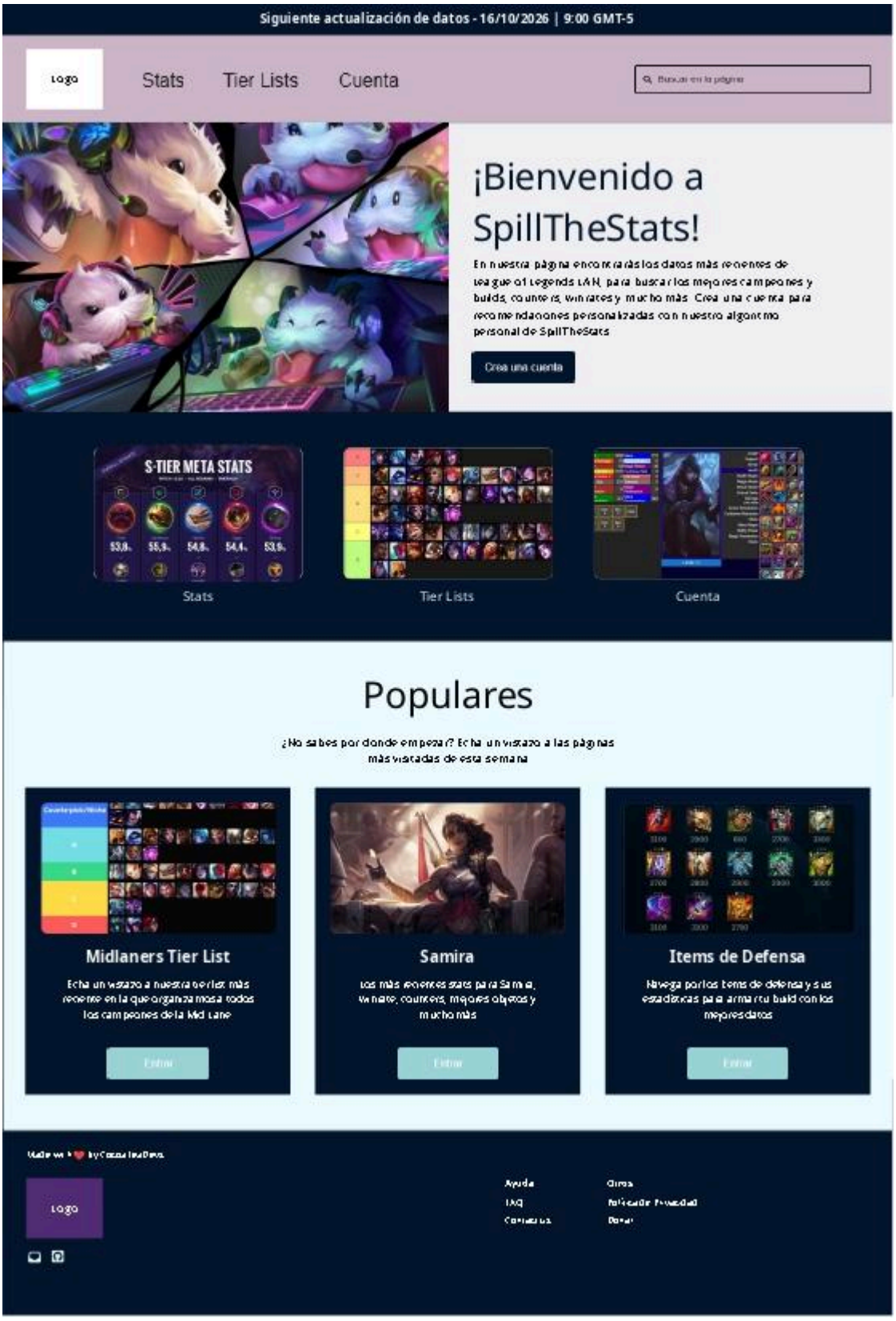
5. Architecture Overview (JAMstack, serverless)



RateLimiter counts Riot requests and activates the optimization protocol at ~80% of the limit.

- **Frontend**: React + Vite + Bootstrap 5, built as a static site and served by Vercel's CDN.
- **Serverless functions** (Vercel) run on every user request: link Riot ID, analyze matches, and the administrator's patch sync. Secrets (Riot key, Supabase service key) live only here.
- **Supabase**: PostgreSQL for data, Auth for login, Storage for files. The browser can read its own data directly thanks to Row Level Security.
- **External APIs**: Riot Games API for account and match data, Data Dragon for static data and images.
- **Rate limits**: analyzing 20 games costs about 22 Riot calls, so the RateLimiter and caching are essential. There are no scheduled jobs; every call happens on demand.

6. Mockups



Home page Mockup: Defines the general visual identity of SpillTheStats and the way we want it to look like.

This Mockup is a simple way to define what we want to achieve with the web app, it might change in the future and it does not reflect everything that will be put on the final deploy of the app, on the github Repository there are also some Logo options and this Mockup on higher quality.

7. Business Model Processes

Process: Request analysis and get recommendations

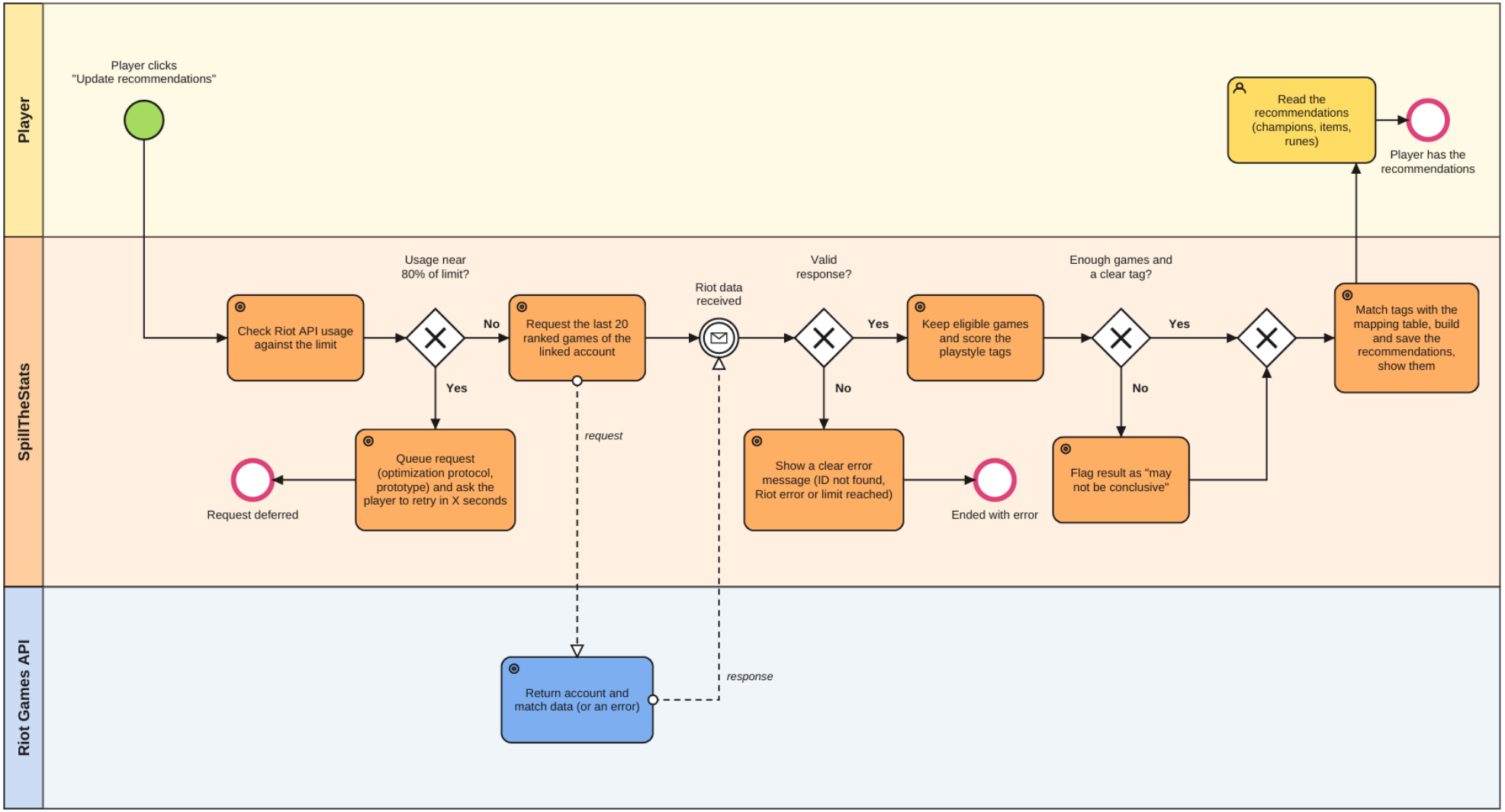
This is the core business process of SpillTheStats. An already registered player with a linked Riot ID asks for an update on their recommendations, and the system turns their latest ranked games into recommendations for the current patch using the defined algorithm. The process is modeled in BPMN 2.0 with three business-level lanes: the Player, SpillTheStats and the Riot Games API.

Item	Description
Trigger	The player clicks "Update recommendations".
Preconditions	The player is registered and logged in, and a Riot ID is already linked (US-01, US-02).
Actors	Player (human), SpillTheStats (system), Riot Games API (external system).
Main outcome	The player reads the recommended champions, items (with situational items) and rune path for the current patch.
Alternative outcomes	**Request deferred**: Riot API usage is near 80% of the limit. **Ended with error**: Riot ID not found, Riot API error or limit reached.
Related rules	BR-02, BR-06, BR-07, BR-08, BR-11 to BR-14, BR-16.

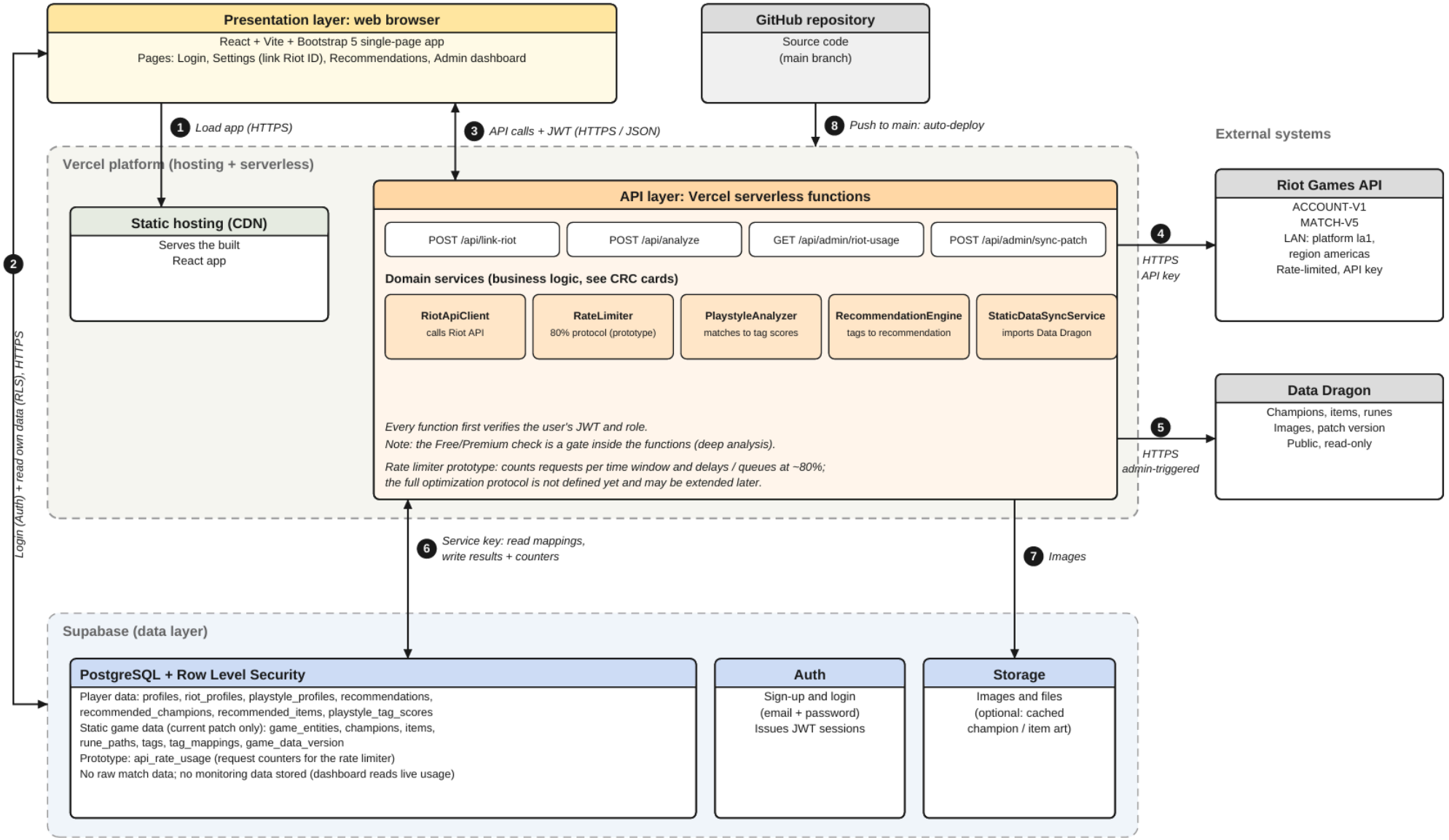
Process description

- The player clicks "Update recommendations" (start event).
- SpillTheStats checks the Riot API usage against the limit. If usage is near 80%, the **optimization protocol** (still a prototype) queues the request and asks the player to retry in X seconds. The process ends as *Request deferred*.
- Otherwise, SpillTheStats requests the last 20 ranked games of the linked account from the Riot Games API, and Riot returns the data or an error.
- If the response is not valid (ID not found, Riot error or limit reached), the system shows a clear error message and the process ends.
- SpillTheStats keeps the eligible games (Ranked Solo/Duo, no remakes, longer than 5 minutes) and scores the playstyle tags.
- If there are few games (e.g., fewer than 10) or no clear dominant tag, the result is flagged as *may not be conclusive*.
- The system matches the tags with the mapping table, builds the recommendations (at least 3 champions, ordered items with situational items, one rune path), saves them and shows them.
- The player reads the recommendations, with the warning when it applies (end event).

Business Process Diagram (BPMN 2.0): Request analysis and get recommendations



8. Architecture Diagram: layers, technologies and communication flows



Architecture Diagram: Explanation

The system is a serverless JAMstack application: a static React frontend, serverless functions as the API, and Supabase as the data layer. The diagram uses a standard layered style, where each box is a layer or a deployable piece and each numbered arrow is a communication flow.

Layers and technologies

Layer	Components	Technology	Responsibility
Presentation	Single-page app: Login, Settings, Recommendations, Admin dashboard	React + Vite + Bootstrap 5, served by the Vercel CDN	Show data and collect user actions
API	Serverless functions: link-riot, analyze, admin/riot-usage, admin/sync-patch	Vercel serverless functions	Verify JWT and role, run the use cases, keep secrets (Riot key, Supabase service key)
Domain services	RiotApiClient, RateLimiter (prototype), PlaystyleAnalyzer, RecommendationEngine, StaticDataSyncService	Code inside the functions	Business logic from the CRC cards
Data	PostgreSQL with RLS, Auth, Storage	Supabase	Persist player data, static data of the current patch, rate-limit counters; login and images
External	Riot Games API, Data Dragon	HTTPS / JSON	Account and match data; static game data and patch version

Communication flows

#	From → To	Protocol and data
1	Browser → Vercel CDN	HTTPS: loads the built React app
2	Browser ↔ Supabase (Auth + PostgreSQL)	HTTPS: sign-up and login (JWT); reads of the user's own data, limited by Row Level Security
3	Browser ↔ Serverless functions	HTTPS / JSON with the user's JWT: link Riot ID, request analysis, admin actions
4	Functions → Riot Games API	HTTPS with the API key (server-side only), through the RateLimiter
5	Functions → Data Dragon	HTTPS, only when an administrator triggers the patch sync
6	Functions ↔ PostgreSQL	Service key: read mappings and static data, write profiles, recommendations and rate-limit counters
7	Functions → Storage	Optional: images
8	GitHub → Vercel	Auto-deploy from the main branch

Notes

- **Rate limiter (prototype):** it counts Riot requests per time window in the api_rate_usage table and delays or queues requests at about 80% of the limit. *It is a first version of the optimization protocol, which is not fully defined yet and may become more complete later.*
- **Admin monitoring:** the dashboard (GET /api/admin/riot-usage) reads usage live; no monitoring data is stored.
- **Free / Premium:** the plan check is a gate inside the functions that decides if the deeper analysis runs.
- **Security:** the Riot and Supabase service keys live only in Vercel environment variables.
- **Scalability**: stateless functions scale with the number of requests; the CDN serves static files; Supabase pooling and indexes cover the main lookups; rate-limit state is shared in the database. The real bottleneck is the Riot rate limit, handled by the RateLimiter.

9. References

- *This document was made with the help of Claude's AI; everything was reviewed and not pasted carelessly in order to achieve a proper understanding of the contents.*
- Riot Games. Riot Developer Portal, API documentation, Data Dragon and API policies. https://developer.riotgames.com
- UML Diagrams. Class diagrams overview. https://www.uml-diagrams.org/class-diagrams-overview.html
- Supabase documentation (PostgreSQL, Auth, Row Level Security). https://supabase.com/docs
- Lucidchart, How to draw architectural diagrams https://www.lucidchart.com/blog/how-to-draw-architectural-diagrams